

Performance Optimization of SLIC Segmentation Algorithm through Concurrent Programming with OpenMP

Marco De Stefano

E-mail address

marco.destefano1@edu.unifi.it

Abstract

Superpixel segmentation is widely used in computer vision pipelines to reduce computational complexity while preserving object boundaries. Among existing methods, SLIC (Simple Linear Iterative Clustering) is particularly attractive due to its linear time complexity and high segmentation quality.

In this project, we present a parallel implementation of the SLIC algorithm in C++ using OpenMP, targeting multi-core CPU architectures. The work focuses on optimizing the two most computationally intensive phases — pixel assignment and centroid update — through data-parallel strategies. We evaluate two memory layouts (Array of Structures and Structure of Arrays) and compare synchronization mechanisms based on atomic operations and reductions.

Experimental results show near-linear scalability with increasing image resolution and significant speedup compared to the sequential baseline, with the reduction-based approach outperforming the atomic strategy for large images. The study highlights the impact of memory layout, synchronization overhead, and data locality on overall performance.

1. Introduction

Superpixel segmentation is a widely adopted preprocessing technique in computer vision pipelines. By grouping perceptually similar pixels into compact regions, superpixels, preserving object boundaries.

Among existing methods, SLIC (Simple Linear Iterative Clustering) is particularly attractive due to its linear computational complexity and high-quality boundary adherence. However, despite its theoretical $O(N)$ complexity, the algorithm remains computationally intensive for high-resolution images, especially when executed sequentially.

This project investigates the parallelization of SLIC using OpenMP on multi-core CPU architectures. We analyze different data layout strategies (Array of Structure vs Structure

of Arrays), evaluate synchronization mechanisms (atomic operations vs reductions), and study their impact on scalability, cache efficiency, and vectorization potential.

The goal is not only to reduce execution time but also to understand the architectural trade-offs that influence performance in data-parallel image processing workloads.

2. Implementation Analysis

Here, the section explains how it is developed and with which one layout data.

2.1. Data Layout

The efficiency of image processing algorithms on modern CPUs relies heavily on data memory layout. To evaluate the impact of spatial locality and vectorization capabilities, two distinct data structure organizations were implemented for representing both the image pixels (N) and the superpixel centroids (K): *Array of Structures* (AoS) and *Structure of Arrays* (SoA).

2.1.1 Array of Structures (AoS) Approach

The Array of Structures (AoS) approach groups all attributes of a single entity into a contiguous memory block. In this layout, both the read-only data heavily accessed during distance calculations and the frequently updated variables are packed together. This structure maximizes spatial locality when all fields of a pixel are accessed simultaneously.

Listing 1. Definition of the Pixel structures in the pure AoS configuration.

```
1 struct alignas(std::
    hardware_destructive_interference_size)
    PixelData_AoS {
2     int x, y;
3     double L, A, B;
4     int label;
5     double distance;
6 };
```

Similarly, superpixels are defined by a structure grouping the centroid coordinates and mean color values.

Listing 2. Definition of the SuperPixel structure in AoS.

```
1 struct SuperPixel_AoS {
2     int label;
3     int centroid_x, centroid_y;
4     float val_L, val_a, val_b;
5 };
```

Talking about the dimension of these structures, we can say that:

- Superpixel $\rightarrow$ this structure has a dimension of 24 bytes.
- Pixel $\rightarrow$ this structure has a dimension of 28 bytes.

2.1.2 Structure of Arrays (SoA) Approach

The `Pixels_SoA` structure acts as a container for pointers to separate arrays (e.g., one array specifically for L values, one for x coordinates, etc.). The memory layout becomes:

$$L = [L_0, \dots, L_{N-1}], \quad x = [x_0, \dots, x_{N-1}], \quad \dots$$

Listing 3. Definition of the Pixel structure in SoA configuration.

```
1 struct Pixels_SoA {
2     double *L, *A, *B;
3     double *distances;
4     int *x, *y;
5     int *labels;
6 };
```

Centroids follow the same logic, facilitating vector access (SIMD) during the update phases.

Listing 4. Definition of the SuperPixel structure in SoA configuration.

```
1 struct SuperPixel_SoA {
2     int* label;
3     int *centroid_x, *centroid_y;
4     double* val_L, *val_a, *val_b;
5 };
```

The SoA layout is theoretically advantageous for SIMD (Single Instruction, Multiple Data) vectorization. It allows the CPU to load multiple values of the same type into vector registers contiguously from memory without loading unnecessary data, maximizing memory bandwidth usage for specific component-wise operations.

2.2. Implementation of the SLIC Algorithm

2.2.1 Sequential Implementation Details

The sequential implementation serves as the baseline for performance evaluation and correctness verification. The execution flow is divided into three main stages: initialization, iterative clustering, and post-processing.

Initialization and Gradient Perturbation The algorithm begins by sampling K superpixel centers on a regular grid with a step size of $S = \sqrt{N/K}$. To avoid placing centroids on edges or noisy pixels, which would result in unstable clustering, a perturbation step is applied. For each center, the gradient is computed in a 3×3 neighborhood using the formula:

$$G(x, y) = \|\mathbf{I}(x+1, y) - \mathbf{I}(x-1, y)\|^2 + \|\mathbf{I}(x, y+1) - \mathbf{I}(x, y-1)\|^2$$

The centroid is then moved to the pixel with the lowest gradient G within this neighborhood.

Iterative Assignment (Localized Search) The core of the algorithm is the assignment step. Unlike standard k-means, which calculates distances between every pixel and every cluster center, our implementation exploits the spatial constraint of superpixels. There are two types of this phase:

- **Pixel Centric:** For each pixel in the image, it calculates the distance with the nine centroids around it (3×3).
- **Centroid Centric:** For each cluster center C_k , the search is restricted to a local window of size $2S \times 2S$ centered at C_k .

The distance D is calculated as a weighted sum of the color proximity (d_{lab}) and spatial proximity (d_{xy}):

$$d_{lab} = \sqrt{(L_i - L_k)^2 + (A_i - A_k)^2 + (B_i - B_k)^2}$$

$$d_{xy} = \sqrt{(x_i - x_k)^2 + (y_i - y_k)^2}$$

$$D = \sqrt{d_{lab}^2 + \left(\frac{d_{xy}}{S}\right)^2} m^2$$

Where m is the compactness factor allowing control over the shape regularity of the superpixels. If D is smaller than the current minimum distance recorded for pixel i , the pixel label is updated to k .

Centroid Update After the assignment pass, the new cluster centers are computed. This step involves iterating through all N pixels, accumulating the coordinate vectors (x, y) and color vectors (L, a, b) for each cluster index, and finally dividing by the count of pixels belonging to that cluster. In the sequential version, this is performed via a simple linear scan of the image arrays, ensuring deterministic behavior.

Enforce Connectivity At the end of the iteration loop, a post-processing step named *Enforce Connectivity* is applied. This ensures that disjoint segments labeled with the same index (stray pixels) are merged into the largest neighboring cluster, guaranteeing that each superpixel forms a single connected component.

2.2.2 Parallel Implementation Details

The parallel implementation leverages **data parallelism** and **not task parallelism**. So, the algorithm's phases are executed in sequential mode but the workload within two of the four phases are distributed across multiple threads using OpenMP.

These two phases are:

- **Iterative Assignment**
- **Update Centroids**

Implementing the parallel version, we explored two distinct synchronization strategies to handle race conditions: one based on **atomic operations** and the other on **parallel reductions**. The parallelization focuses on the two most computationally intensive phases of the algorithm: the *Assignment Step* and the *Centroid Update Step*.

Parallel Assignment As mentioned before, there are two main approaches to implement this phase. We chose the **Pixel-Centric** mode over the traditional **Centroid-Centric** one to overcome two major architectural bottlenecks.

- **First**, the Centroid-Centric approach suffers from **race conditions**: threads processing adjacent centroids will inevitably compete to update the distance of the same pixels located on their overlapping boundaries. Preventing this would require costly synchronization mechanisms, such as locks or atomic operations, severely degrading performance.
- **Second**, the Centroid-Centric mode exhibits poor memory locality. Since each centroid checks all pixels within a $2S \times 2S$ region, the algorithm must continuously jump across image rows. This breaks the sequential memory access pattern, leading to high latency and frequent cache misses.

Conversely, in the Pixel-Centric approach, each thread is strictly assigned a subset of pixels. The thread iterates sequentially through the image memory (stride-1 access), evaluating the 9 neighboring centroids for each pixel. This drastically reduces cache misses. Furthermore, since a pixel's distance and label are written exclusively by its owning thread, there are no read-modify-write conflicts on shared structures.

The Tile technique is implemented in the **Assignment Phase**.

Why here? And why not in the **Update Centroids**?

The fact is that in the Update Centroid Phase, it purposely avoided the tiling strategy since this phase involves

a single pass over the pixel data (*Streaming Access Pattern*) with no data reuse, a standard Linear Scan is optimal.

Meanwhile, with the tiling in the Assignment Phase, we can load a tile of pixels and also the centroids used in the cache and throw away these when the next tile is elaborated.

Parallel Update Centroid (Atomic vs. Reduction) The update phase iterates through all pixels to accumulate the color components (L, a, b) and spatial coordinates (x, y) for the subsequent centroid recalculation. To store these intermediate sums, we adopted a **Structure of Arrays (SoA)** layout, consisting of six independent arrays of size K (one for each attribute plus a counter for the number of pixels).

To optimize memory performance on multi-core architectures, we implemented a **NUMA-aware initialization** strategy. By initializing these cumulative arrays within a parallel region, we exploit the **First-Touch Policy**: this ensures that the memory pages are physically mapped to the NUMA node closest to the thread performing the initialization, significantly reducing remote memory access latency during the accumulation process.

However, this phase introduces a critical synchronization challenge. Since multiple threads may simultaneously process different pixels belonging to the same cluster, they would attempt to update the same memory location in the global cumulative arrays, leading to potential **race conditions**. To address this, we evaluated two distinct strategies: atomic operations and OpenMP reductions.

We implemented two approaches to address this:

- **Atomic approach**: Threads share the global array of centroids. When a thread processes a pixel, it adds its values to the corresponding centroid accumulator using `#pragma omp atomic`.

- *Pros*: Minimal memory overhead.
- *Cons*: High contention (overhead) when many pixels belong to the same superpixel, causing serialization of operations.

- **Reduction approach (`reduction_parallel = true`)**: Each thread allocates a private copy of the centroid accumulators (size K). Threads perform the accumulation on their private copies independently, eliminating race conditions during the loop. At the end of the parallel region, these private copies are merged (reduced) into the global centroid array.

- *Pros*: Zero contention during the loop, typically faster for large images.

- *Cons*: Higher memory consumption, as the centroid structures are replicated T times (where T is the number of threads).

Once the accumulation phase is complete, we compute the average for each cluster by dividing the accumulated sums by the number of pixels. We fully parallelized this final step and explicitly forced SIMD vectorization using OpenMP.

Enforce Connectivity Regarding the selection of the algorithmic phases to parallelize, we decided to leave the Initialization and the Enforce Connectivity steps strictly sequential.

Amdahl's Law states that: *"The overall performance improvement gained by optimizing a single part of a system is limited by the fraction of time that the improved part is actually used."*

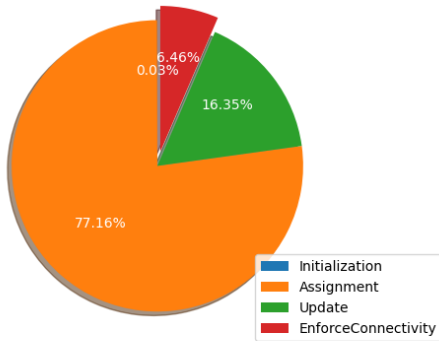


Figure 1. Initialization and Enforce Connectivity account for a minor fraction of the total execution time.

Using Amdahl's Law, the maximum theoretical speedup is defined as:

$$Speedup_{max} = \frac{1}{(1 - P) + \frac{P}{N}}$$

where:

- $(1 - P)$: fraction of the strictly sequential (non-parallelizable) part.
- P : fraction of the parallelizable part.
- N : number of available cores/threads.

In our implementation, the sequential remaining part (primarily the Enforce Connectivity phase) accounts for approximately 6% of the total execution time ($(1 - P) = 0.06$). For our target Apple Silicon architecture featuring

$N = 8$ cores, the maximum achievable speedup is strictly bounded:

$$Speedup_{max}(8) = \frac{1}{0.06 + \frac{0.94}{8}} = \frac{1}{0.06 + 0.1175} \approx 5.6 \times$$

Even if we had a machine with an infinite number of cores ($N \rightarrow +\infty$), the theoretical asymptotic limit would still cap at:

$$Speedup_{max}(\infty) = \frac{1}{0.06} \approx 16.6 \times$$

The Connectivity Enforcement stage is a sequential approach based on **Breadth-First Search** (BFS). Unlike the distance calculation and centroid update phases, which are inherently data-parallel, connectivity analysis involves strong spatial data dependencies (i.e., a pixel's label depends on the outcome of its neighbors' traversal).

Parallelizing this post-processing step would require complex synchronization mechanisms to handle irregular memory access patterns, introducing massive overhead. Given its minimal impact on the total execution time, parallelizing this step was deemed highly inefficient and theoretically unjustified according to Amdahl's framework.

2.3. Hardware Specifications

The experimental evaluation was conducted on an Apple Silicon System-on-Chip (SoC) featuring an ARM-based architecture. The processor includes a total of 8 physical cores without Simultaneous Multithreading (SMT), meaning the number of physical cores equals the number of logical threads.

The cores are divided into two distinct clusters:

- **4 Performance Cores (P-Cores):** Equipped with:
 - 192 KB L1 Instruction Cache
 - 128 KB L1 Data Cache
 - shared 12 MB L2 Cache
- **4 Efficiency Cores (E-Cores):** Equipped with:
 - 128 KB L1 Instruction Cache
 - 64 KB L1 Data Cache
 - shared 4 MB L2 Cache

The system is equipped with 8 GB of unified memory and an architectural detail for the memory access pattern analysis is the **cache line size of 128 bytes on the Performance Cores (P-cores)**, which is double the standard 64 bytes typically found in x86 architectures and in the local Efficiency Cores (E-cores). This wider cache line significantly enhances spatial locality on the P-cores.

Consequently, this architectural asymmetry introduces a noticeable performance discrepancy between the E-cores and the P-cores when executing memory-bound operations.

2.4. Software Environment and Setup

The benchmarking suite was executed and profiled on the macOS operating system (Apple Silicon architecture). The project was built using **CMake** as the build system. The default **Apple Clang (LLVM)** compiler was utilized. The following key libraries and architectural configurations were integrated into the environment:

- **OpenMP:** Since Apple Clang does not natively bundle OpenMP runtime libraries, the LLVM OpenMP library (`libomp`) was located via the Homebrew package manager. Specific compiler and linker flags (`-Xpreprocessor -fopenmp`) were explicitly injected during the build phase to enable `#pragma omp` directives for shared-memory multi-processing.
- **OpenCV:** The Open Source Computer Vision Library was strictly utilized for peripheral tasks:
 - image I/O operations
 - initial color space conversion (BGR to CIELAB)
- **Aggressive Compiler Optimizations:** The `-O3` flag (enabling aggressive loop unrolling and auto-vectorization), the `-ffast-math` flag was introduced to relax strict IEEE 754 floating-point compliance, granting the compiler more freedom to optimize the heavy mathematical distance calculations. Furthermore, architecture-specific tuning (`-mcpu=apple-m1`) was applied to fully exploit the underlying hardware capabilities.

3. Experiments Analysis

This section presents the experimental validation of the implemented algorithm. Each subsequent subsection details a specific experiment and discusses the obtained results.

3.1. Complexity and Scalability Analysis

As a preliminary assessment, we verified how the algorithm's performance scales with increasing image resolution.

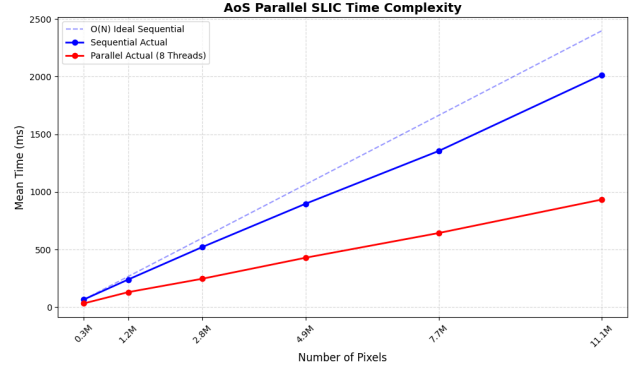


Figure 2. The gray line is the equation $y=x$

As shown in Figure 2, the mean execution time increases linearly with resolution. The benchmark measured the SLIC execution time starting with a base resolution of 640x480, scaling the image up to six times its original size, and logging the results to "benchmark_complexity.csv". In the plot, the red curve represents the execution time of the parallel version. As expected, the parallel implementation demonstrates significantly lower execution times compared to the sequential one.

Resolution	Mean time seq.	Mean time parall.	Speed-up
×1	70.90ms	33.99ms	x2.09
×2	242.00ms	112.93ms	x2.14
×3	531.11ms	234.18ms	x2.27
×4	901.82ms	428.87ms	x2.10
×5	1363.40ms	584.65ms	x2.33
×6	1976.22ms	910.95ms	x2.17

Table 1. Comparison between sequential and parallel execution times with corresponding speed-up

3.2. Experiment on the number of Threads

To determine the optimal number of threads for this algorithm, we evaluated the execution time using a static scheduler while progressively increasing the thread count. The following graph illustrates the Speed-Up achieved relative to the number of threads.

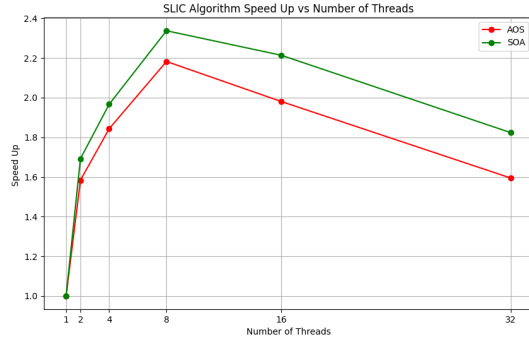


Figure 3. Speed-up varying the number of threads.

In Figure 3, the maximum speedup is achieved exactly at eight threads, after which performance steadily declines. This behavior perfectly mirrors the host machine’s architecture, which features exactly eight physical cores without Hyper-Threading. Consequently, spawning more than eight threads triggers **Thread Oversubscription**, introducing severe performance penalties:

1. **Context Switching & Cache Thrashing:** When the operating system is forced to multiplex more threads than available physical cores, the CPU wastes cycles saving and restoring thread states instead of performing useful computations. Worse, this continuous switching leads to *Cache Thrashing*: when a thread is suspended, its cached data is often evicted by the incoming thread. Upon resuming, it suffers massive cache misses, forcing expensive data reloads from the main memory.
2. **AoS Memory Bloat (The Padding Penalty):** As shown in Figure 3, the AoS curve (red line) degrades much more steeply than the SoA curve (green line) at 16 and 32 threads. To prevent the well-known Read-Write *False Sharing* bottleneck, our pure AoS implementation explicitly aligned the pixel structures using `alignas`. While this successfully isolates each pixel into its own cache line, it introduces massive *Memory Bloat* (padding). During the continuous context switches caused by oversubscription, the CPU wastes precious memory bandwidth reloading Cache Lines filled mostly with empty padding, severely reducing the effective L1/L2 cache capacity and amplifying the penalties of Cache Thrashing. Conversely, the dense and contiguous memory layout of SoA remains highly resilient under heavy thread contention.

Based on these architectural constraints, all subsequent experiments will be parallelized using exactly eight threads.

3.3. Evaluation of OpenMP Scheduling Policies and Chunk Sizes

To maximize the parallel efficiency of the SLIC algorithm, we evaluated the impact of different OpenMP scheduling policies (`static`, `dynamic`, and `guided`) and varying chunk sizes (1, 10, 50, 100) across different resolutions, memory layouts, and synchronization mechanisms.

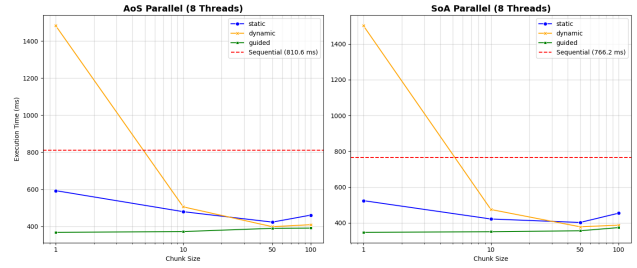


Figure 4. Execution Time of parallelization with different types of schedules and chunk sizes, using reductions.

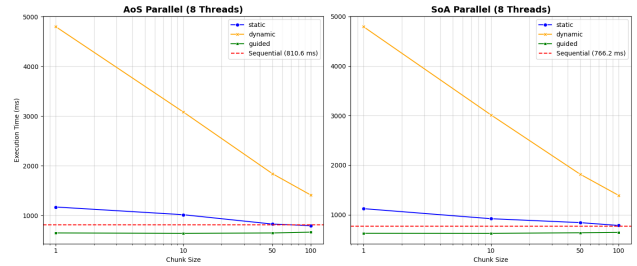


Figure 5. Execution Time of parallelization with different types of schedules and chunk sizes, using atomics.

Analysis of the experimental data reveals three consistent architectural behaviors regarding workload distribution:

- **High Overhead in Dynamic Scheduling:** The `dynamic` schedule exhibited the poorest performance, particularly with a chunk size of 1. The runtime overhead of assigning microscopic tasks (a single loop iteration) to threads far outweighs the computational cost of the distance calculation itself. Performance only recovers at larger chunk sizes (e.g., 100) where this synchronization tax is effectively amortized.
- **Rigidity of Static Scheduling:** The `static` schedule demonstrates flat execution times across all chunk sizes since the workload division is pre-computed entirely before loop execution. However, it fails to dynamically account for the physical heterogeneity of the Apple Silicon architecture (P-Cores vs. E-Cores), leading to physiological load imbalances.

- **Superiority of Guided Scheduling:** Across almost all configurations, the `guided` policy yielded the lowest execution times. By starting with large blocks that proportionally decrease in size, it perfectly marries the low-overhead benefits of `static` scheduling with the adaptive load balancing required by the asymmetric hardware, remaining highly efficient even at minimal chunk sizes.

As a final synthesis of these architectural tuning steps, Figure 6 illustrates the best-performing configurations across all resolutions. Ultimately, the absolute peak performance of the entire study is achieved by combining the **SoA layout with reductions, a guided scheduler, and a chunk size of 1**.

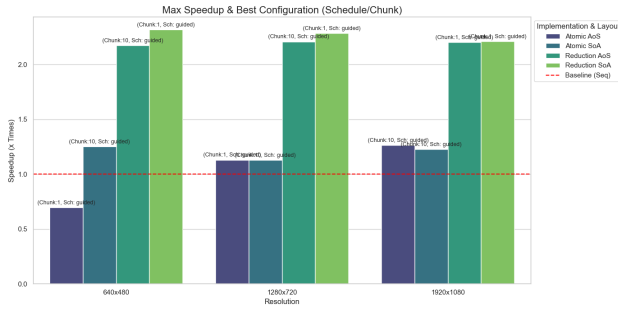


Figure 6. Plot taking the max configuration for all implementations and all resolutions.

Regarding the memory layout, calculating the SLIC distance requires accessing all spatial and color components (L, a, b, x, y) simultaneously for each pixel. While the **AoS layout** intuitively provides optimal spatial locality for this specific read-heavy access pattern, the graph shows that the **SoA layout** slightly edges it out. The densely packed SoA layout maximizes Cache L1/L2 capacity utilization, yielding the highest overall peak performance.

Furthermore, we can see a degradation on the image 640x480 on Atomics synchronization and layout AoS caused by **True Sharing** on the centroid accumulators: since threads process spatially adjacent pixels that map to the same centroids, they concurrently attempt to update the exact same memory locations. This high contention floods the CPU pipeline with simultaneous atomic lock requests, leading to massive execution stalls.

3.4. Tiled or Not Tiled

As a final experiment to determine the optimal configuration for this algorithm, we evaluated the impact of loop tiling (or blocking). Tiling is a well-known optimization technique that aims to improve cache utilization by loading a specific block of pixels into the L1/L2 cache, elaborating these data without continuously fetching from the main memory (RAM).

The tile size used in these experiments matches the dimension of the local search area defined by the SLIC algorithm, which is $2S \times 2S$. Despite the theoretical benefits, we expected a degradation in performance for the tiled implementation.

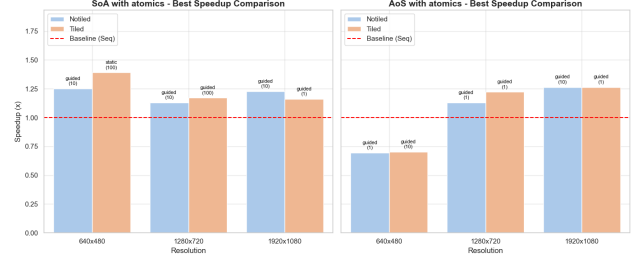


Figure 7. Tiled vs Not Tiled performance using Atomic operations.

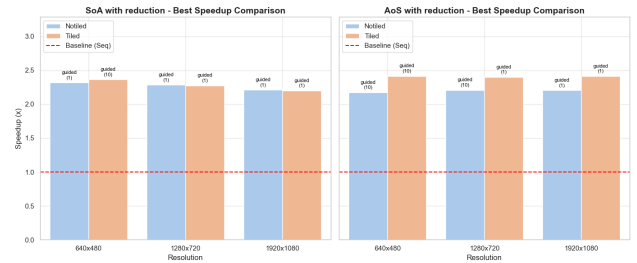


Figure 8. Tiled vs Not Tiled performance using Reduction operations.

Before analyzing the direct impact of Tiling, it is important to note from Figure 7 that, even with the optimized layout, the implementation with atomic synchronization performs significantly worse than the sequential baseline because of the **True Sharing**.

Architectural Impact of Tiling on Data Layouts As illustrated in Figure 8, the application of loop tiling yields contrasting effects depending on the underlying memory layout:

- **AoS Resilience under Tiling:** The Tiled AoS implementation has a bigger improvement compared to the SoA layout. Because in the Assignment Phase, using Tiling we have the nine local centroids fixed in the cache, meanwhile the pixels act as "tourists". So, there is an efficient recycle of these resources: since both centroids and pixels are stored as contiguous structures, a single cache line fetch loads all attributes (L, a, b, x, y) at once, allowing the CPU to quickly compute the distances without stalling.
- **SoA Behavior:** Conversely, the Tiled SoA approach collapses precisely because of its structural fragmentation. In a pure SoA implementation, not only are

the "tourist" pixels scattered across five separate arrays (L, a, b, x, y), but the centroids are separated as well. When processing a 2D tile, the CPU is forced to manage up to ten independent memory streams simultaneously (five for the jumping pixels and five for the centroids). Because the tile requires reading short segments of a row and then jumping to the next, the access pattern becomes highly strided.

4. Conclusions and Future Works

Our extensive experimental campaign demonstrates that parallelizing the SLIC algorithm on a heterogeneous multi-core CPU can achieve a peak speedup of approximately $2.41\times$. This result is the culmination of a rigorous architectural tuning process that highlighted critical interactions between memory layouts, synchronization primitives, and hardware behavior. Contrary to initial expectations, the **AoS (Array of Structures) layout combined with loop tiling** emerged as the absolute best configuration for peak performance. While the SoA layout is traditionally favored for linear scans, the AoS structure demonstrated superior resilience and efficiency when processing data in 2D blocks. This is due to a "perfect marriage" between spatial and temporal locality: the tiling keeps the nine local centroids pinned in the L1/L2 caches, while the AoS layout ensures that each memory fetch for a "tourist" pixel brings all its attributes (L, a, b, x, y) into the cache simultaneously. This synergy minimizes cache thrashing and maximizes throughput. To effectively harness this parallel power, **OpenMP reductions** proved structurally superior to atomic operations, completely eliminating the massive True Sharing contention observed during the centroid update phase. Furthermore, the **guided scheduler** (with a chunk size of 1 or 10) was fundamental to handling the hardware asymmetry of Apple Silicon's P-Cores and E-Cores, dynamically balancing the workload without the excessive runtime overhead of purely dynamic policies. The analysis also revealed that the **SoA (Structure of Arrays)** layout, though highly efficient for standard linear scans thanks to the hardware prefetcher, is less effective under tiling conditions. Scattering both pixels and centroids across ten separate arrays forces the CPU to manage multiple independent memory streams during strided 2D accesses. This fragmentation introduces an architectural bottleneck that prevents the SoA layout from scaling as effectively as the AoS when the access pattern deviates from a simple linear scan.

Future Works Future developments could investigate a *Centroid-Centric* parallelization approach to further reduce synchronization overhead. Additionally, the highly parallel nature of the distance calculations suggests that porting the algorithm to the GPU using the **CUDA Framework**

could yield massive performance gains, exploiting high-bandwidth memory and thousands of concurrent threads to surpass the limitations of CPU-based architectures.

5. Future Implementations

After seen what is the best configuration to speed up this algorithm, we could think of future implementation from this baseline. Infact, for this project we have used a Pixel Centric behavior but we could try with the Centroid Centric one. Furthermore, we could try to implement a parallelized version with the use of CUDA Framework.

6. Bibliography

References

- [1] R. Achanta, A. Shaji, K. Smith, A. Lucchi, P. Fua, and S. Süsstrunk, "SLIC Superpixels Compared to State-of-the-Art Superpixel Methods," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 34, no. 11, pp. 2274–2282, Nov. 2012.
- [2] OpenMP Architecture Review Board, "OpenMP Application Programming Interface," Version 5.2, Nov. 2021. [Online]. Available: <https://www.openmp.org>
- [3] Bradski, G. *The OpenCV Library*. Dr. Dobb's Journal of Software Tools, 2000.
- [4] Harris, C. R., Millman, K. J., van der Walt, S. J., et al. *Array programming with NumPy*. Nature, 585(7825):357–362, 2020.
- [5] McKinney, W. *Data Structures for Statistical Computing in Python*. In Proceedings of the 9th Python in Science Conference, pages 56–61, 2010.
- [6] Hunter, J. D. *Matplotlib: A 2D graphics environment*. Computing in Science & Engineering, 9(3):90–95, IEEE Computer Society, 2007.